Week 1 Deliverable: Project Management Plan

Student Name: Katleho Ernest Borotho

Student Number: 202322762

1. Project Management Plan

This project management plan covers the first phase of the Moderated Learning Management System project. The project will be developed over seven weeks using the Waterfall model, with V-Model verification and validation. Week 1 planning work consists of the feasibility study, Work Breakdown Structure, project schedule and risk register.

The system will be implemented as a C++17 desktop application with a simple graphical user interface (GUI), SQLite persistent storage, CMake for building, Git for version control, an automated unit-testing framework and a static analyser. Version 1.0 will target a single school computer and will not require Internet connectivity for normal operation. A second laptop will be used only as an independent deployment test machine to verify that the packaged application can run outside the development environment.

2. Feasibility Study

Feasibility Area	Assessment
Technical feasibility	The project is technically feasible. The required technologies are suitable for a local desktop system. Version 1.0 will target a single school computer and will use C++17, a simple desktop GUI and SQLite. The GUI will contain clear role-based screens, forms, buttons and tables, while the business logic and database access will remain separate from the interface. The system will not require Internet connectivity for normal operation. The main technical risks are GUI framework integration, password protection, database access, role-based access control, audit logging, performance on older computers and release-package compatibility on another computer.
Operational feasibility	The project is operationally feasible. A simple GUI is appropriate because the system will be used by students, lecturers, Heads of Department and an administrator, and the principal requires the system to be simple for non-technical teaching staff. Each role will have a clear dashboard showing only the functions that the user is allowed to perform. Version 1.0 will be used on a single school computer and will not depend on Internet access, which supports the school environment where Internet connectivity is unreliable.
Economic feasibility	The project is economically feasible for this assignment. No new development hardware is planned because existing laptops will be used for development and deployment verification. The assignment already specifies commonly available development tools such as C++17, CMake, SQLite, Git, a testing framework and a static analyser. No unsupported monetary estimate is included because the assignment brief does not provide labour rates, hardware prices or a school budget.
Legal feasibility	The project is legally feasible if learner information and assessment records are protected according to applicable legislation and school policy. The system will use authentication, role-based access, protected passwords and an audit log for mark changes. The assignment brief does not identify a specific data-protection law, so the project will not claim compliance with a named law until the required legal analysis is completed later in the project.

Feasibility Area	Assessment
Schedule feasibility	The seven-week schedule is feasible but tight. Requirements and design must be completed on time because implementation cannot begin until Gate 2 has been passed. The GUI will remain simple so that interface development does not consume time needed for the core LMS rules, database, security and testing. Weekly progress checks and strict scope control will be used to protect the schedule.

2.1 Feasibility Recommendation

The project will proceed. A simple desktop GUI has been selected instead of a console interface because it better supports the usability needs of the intended users while remaining within the assignment scope. Version 1.0 will be deployed on a single school computer, will not require Internet connectivity for normal operation, and will store data locally in SQLite. The second laptop will be used only to verify the deployment package. High complexity features that are outside the assignment scope, including online payment, video conferencing, automated grading, plagiarism detection and web or mobile interfaces, will not be added.

3. Work Breakdown Structure (WBS)

WBS ID	Work Package	Time	Output
1.0	Project Planning and Management	Week 1	Project management plan
1.1	Feasibility study	Week 1	Technical, operational, economic, legal and schedule feasibility
1.2	Work breakdown and project schedule	Week 1	WBS, Gantt chart, dependencies, milestones and critical path
1.3	Risk management	Week 1	Initial risk register
2.0	Requirements Engineering	Weeks 1-2	Stakeholder analysis, SRS and use case model
2.1	Stakeholder analysis and conflict resolution	Weeks 1-2	Stakeholder interests, conflicts and resolutions
2.2	Software Requirements Specification	Weeks 1-2	Functional and measurable non-functional requirements
2.3	Use case model	Week 2	Use case diagram and three fully dressed use cases
2.4	Gate 1 review	End of Week 2	Requirements review
3.0	System Design	Weeks 2-3	Software design document
3.1	Architecture design	Weeks 2-3	Architecture and component diagram
3.2	Detailed class design	Weeks 2-3	Class diagram, design patterns and SOLID justification
3.3	Behavioural modelling	Weeks 2-3	State, activity and sequence diagrams

WBS ID	Work Package	Time	Output
3.4	Database design	Week 3	3NF ERD, data dictionary and SQL schema
3.5	GUI/interface design	Week 3	Login screen and role-specific dashboards/forms
3.6	Gate 2 review	End of Week 3	Design approval before implementation
4.0	Implementation	Weeks 4-5	C++17 desktop application
4.1	Project and build structure	Week 4	CMake project and source structure
4.2	GUI implementation	Weeks 4-5	Login and role specific screens
4.3	Business logic and access control	Weeks 4-5	LMS rules and role permissions
4.4	Persistence and security	Weeks 4-5	SQLite data access, password protection and audit log
4.5	CI and code review	Weeks 4-5	Automated build/test pipeline and code review evidence
5.0	Verification and Validation	Weeks 5-6	Test plan, automated tests, results and defect log
5.1	Test strategy	Week 5	V-Model test levels and test techniques
5.2	Unit and integration testing	Weeks 5-6	Automated tests and coverage result
5.3	System and acceptance testing	Week 6	System tests, acceptance tests and defect log
5.4	Gate 3 review	End of Week 6	Verification and validation review
6.0	Configuration Management and QA	Weeks 4-6	Repository, CI, static analysis, metrics and review record
7.0	Deployment	Week 6	Deployment diagram, installation guide, user manuals and v1.0 release for the single-computer target
7.1	Second-laptop deployment test	Week 6	Verify packaged v1.0 on a second laptop; the laptop is a test machine, not part of the operational LMS architecture
8.0	Maintenance	Week 7	CR1 analysis, v1.1 and regression testing
9.0	Completion	Week 7	Professional issues, final report, appendices and viva preparation

4. Project Schedule and Gantt Chart

The project is scheduled according to the seven-week phase structure provided in the assignment. The symbol X shows planned work and M shows a milestone. In this schedule, an activity is marked as critical when delaying it would delay a required successor activity, prevent a phase gate from being reached on time, or delay the final completion of the project. Activities marked as non-critical are still required, but they have some scheduling flexibility or can be carried out in parallel without immediately delaying the next major phase.

Activity	Critical	W1	W2	W3	W4	W5	W6	W7
Project planning & management	Yes	X						
Stakeholder analysis & conflict resolution	Yes	X	X					
Software Requirements Specification (SRS)	Yes	X	X					
Use case model	Yes		X					
Gate 1 - Requirements review	Yes		M					
Architecture design	Yes		X	X				
Class design & design patterns	Yes		X	X				
Behavioural UML models	Yes		X	X				
Database design	Yes			X				
GUI/interface design	Yes			X				
Gate 2 - Design review	Yes			M				
Git, CMake & CI setup	No				X			
C++ implementation including GUI	Yes				X	X		
SQLite persistence & security	Yes				X	X		
Configuration management & QA	No				X	X	X	

Activity	Critical	W1	W2	W3	W4	W5	W6	W7
Test strategy	No					X		
Unit & integration testing	Yes					X	X	
System & acceptance testing	Yes						X	
Gate 3 - V&V review	Yes						M	
Deployment & release v1.0	Yes						X	
Second-laptop deployment test	No						X	
Maintenance CR1 & release v1.1	Yes							X
Final report & viva preparation	No						X	X

4.1 Dependencies and Milestones

Activity/Milestone	Depends On	Reason
Project planning	None	Establishes the project baseline.
Requirements engineering	Project planning	Defines what the system must do before detailed design.
Gate 1	Requirements package completed	Requirements are reviewed before the design phase is completed.
Architecture, class, behavioural and database design	Requirements sufficiently defined	The design is based on the agreed requirements and must be complete for Gate 2.
GUI/interface design	Use cases and role requirements defined	The GUI is part of the chosen system design and must be ready before Gate 2.
Gate 2	System design completed	Implementation starts only after this milestone is passed.
Git, CMake and CI setup	Gate 2 passed	Supports controlled implementation and automated builds/tests but is scheduled with some flexibility in Week 4.
Implementation	Gate 2 passed	Implements the approved design, including the GUI, business logic, persistence and security.

Activity/Milestone	Depends On	Reason
Configuration management and QA	Implementation artefacts available	Runs alongside implementation and testing to provide version-control, static-analysis, metrics and review evidence.
Unit and integration testing	Implemented components available	Checks components and their interaction as implementation progresses.
System and acceptance testing	Integrated system available	Validates the complete system against requirements and use-case behaviour.
Gate 3	Required V&V evidence completed	Testing evidence is reviewed before release evidence is finalised.
Deployment v1.0	System validated	The release package, manuals and installation guide are prepared for the single-school-computer target.
Second-laptop deployment test	v1.0 package prepared	Uses a second laptop only to verify that the packaged application runs outside the development environment.
Maintenance v1.1	v1.0 released	CR1 is analysed, implemented and regression tested.
Final report and viva preparation	Project artefacts available	Runs in parallel with late project work and uses evidence collected throughout the project.
Completion	Required deliverables completed	The complete report, repository evidence and viva preparation are ready for submission.

4.2 Critical Path

The phase-level critical path is, in order: Project Planning, Requirements Engineering, Gate 1, System Design (including GUI/interface design), Gate 2, Implementation, Verification and Validation, Gate 3, Deployment v1.0, Maintenance v1.1, and Completion. GUI/interface design is critical because it must be completed as part of the approved design before implementation begins. Configuration management and quality assurance, the second-laptop deployment test, and final report/viva preparation are scheduled in parallel with critical work and are not marked as critical in this Week 1 schedule. They are still required project activities. Because the assignment gives phase weeks rather than task durations in days, this is a phase-level critical path and can be refined if detailed durations are introduced later.

5. Risk Register

Likelihood and impact are rated from 1 (very low) to 5 (very high). Exposure is calculated as Likelihood x Impact. The risk register will be reviewed at Gate 1, Gate 2 and Gate 3.

ID	Category	Risk	L	I	Exposure	Mitigation	Owner
R1	Project	Schedule slippage in the fixed seven-week project	4	5	20	Check progress weekly, protect gate dates, complete critical work first and prevent scope growth.	Student developer
R2	Project	Conflicting or unclear stakeholder requirements cause rework	4	4	16	Resolve conflicts during engineering requirements, write verifiable requirements and review them at Gate 1.	Student developer
R3	Technical	GUI framework, C++17, CMake or SQLite integration problems delay implementation	3	4	12	Verify the development toolchain early, keep the GUI simple, minimise dependencies and document build/setup steps.	Student developer
R4	Technical	Security weakness exposes passwords or assessment records	3	5	15	Use protected password storage, role-based access, parameterised queries, input validation and security tests.	Student developer
R5	Technical	The application performs poorly on older computers	3	4	12	Keep the GUI lightweight, use efficient database queries and test the application with representative data.	Student developer
R6	Technical/Project	Unreliable Internet interrupts remote Git or CI access	3	3	9	Commit locally, keep local backups and push changes when connectivity is available.	Student developer
R7	People	Illness or other academic work reduces available project time	2	5	10	Work ahead of deadlines, prioritise critical-path work and commit progress frequently.	Student developer
R8	Project	Source code, report or database files are lost or corrupted	2	5	10	Use Git, regular backups and tagged releases; keep an additional copy on a second device.	Student developer
R9	Project	Gate 2 is delayed or the design is not accepted	2	5	10	Check design completeness against the brief before review and resolve missing items before the gate.	Student developer

ID	Category	Risk	L	I	Exposure	Mitigation	Owner
R10	Project	Extra features cause scope creep	2	4	8	Keep the system within the stated scope and do not add web/mobile interfaces, payment, video conferencing, automated grading or plagiarism detection.	Student developer
R11	Technical/Deployment	The packaged application does not run correctly on a computer other than the development laptop	2	3	6	Prepare the release package before final deployment, include required runtime dependencies, test v1.0 on the second laptop and correct any installation problems found.	Student developer

5.1 Risk Review

At each phase gate, the likelihood and impact of every open risk will be reviewed. New risks will be added when identified, completed mitigations will be recorded and risks that no longer apply will be marked closed. Previous entries will be retained so that changes in project risk can be traced.

6. Project Decisions

Decision	Selected Approach
Development model	The project will follow the Waterfall model required by the assignment, with V-Model verification and validation.
User interface	The system will use a simple desktop GUI. The interface will use clear role-specific screens and forms and will remain lightweight enough for older computers.
Deployment approach	Version 1.0 will target a single school computer. The desktop LMS and local SQLite database will not require Internet connectivity for normal operation. The local network option allowed by the assignment is not part of the selected v1.0 implementation.
Architecture direction	The GUI, business logic and data-access responsibilities will be separated. SQLite will only be accessed through the data-access layer.
Deployment testing	The packaged v1.0 application will be installed and tested on a second laptop to confirm that it can run independently of the development environment. The second laptop is only a deployment verification machine and is not part of the operational LMS architecture.
Scope control	Only the LMS functions in the assignment will be developed. Features that are explicitly out of scope will not be added.
Security	Authentication, role-based permissions, protected passwords and mark-change auditing will be treated as required system constraints from requirements through testing.
Traceability	Requirement, use case, design and test identifiers will be kept consistent so that the final traceability matrix can be completed directly from project artefacts.

7. Conclusion

The Moderated Learning Management System is feasible within the seven-week project period if the scope is controlled and the phase gates are met on time. The system will be developed as a simple C++ desktop GUI application with local SQLite storage and will not require Internet connectivity for normal operation. Version 1.0 will target a single school computer, while a second laptop will be used only for deployment verification. The project schedule, WBS and risk controls provide the baseline that will be used for the remaining phases.

8. References

CS4433 Software Engineering lecture notes. Chapter 2 - Software Processes.

CS4433 Software Engineering lecture notes. Introduction to Software Engineering.